If you want, next I can help you define a concrete complexity metric and a simple rule engine you can implement in code for this hackathon.

Yes, let's do that. I'll give you a **simple, implementable complexity metric** and a **rule engine structure** you can code quickly.

1. Complexity Metric (easy to compute)

Define an overall **Topology Complexity Score (TCS)** as a weighted sum of basic graph features:

$$\text{TCS} = w_1 \cdot E + w_2 \cdot H + w_3 \cdot F + w_4 \cdot C$$

Where (all computable from the CSVs):

- E: total number of channels (edges).
- H: average hop count between producer and consumer QMgrs (shortest path length in graph).
- F: max fan-out from any node (max out-degree among QMgrs).
- C: number of cycles in the QMgr graph.

Pick simple weights like $w_1 = 1, w_2 = 2, w_3 = 1, w_4 = 3$ to emphasize hops and cycles.
Your story: "We reduce TCS from X to Y while respecting all constraints and keeping connectivity."

2. Rule Engine Structure

Represent rules as **pure functions** over your graph / tables. For each rule:

- Input: as-is model (QMgrs, queues, channels, app-QMgr mapping).
- Output: proposed changes + explanation.

Example core rules aligned to the screenshot:

1. **One QMgr per App**
   - If an AppID connects to multiple QMgrs → pick primary, reroute others via remote queues and channels to that QMgr, then drop illegal direct connections.
2. **Producers write to local queues**

- If a producer is mapped to a remote queue directly on another QMgr → create local queue + remoteQ + XMITQ on its own QMgr, connect via sender/receiver channels.

3. **Consumers read from local queues only**
   - If a consumer is attached to a remote queue → create local queue on its QMgr, connect via remoteQ/XMITQ pattern.
4. **Channel consolidation rule**
   - If there are multiple sender channels between the same QMgr pair → consolidate into one, adjust routes to use it.
5. **Redundant object cleanup**
   - If a queue / channel has no producer and no consumer after refactoring → mark for deletion.
6. **Cycle breaker rule**
   - If a cycle exists between QMgrs → pick a non-critical channel in that cycle and reroute or remove it while preserving necessary paths.

3. Implementation Sketch (Python-friendly)

Data model (Pandas + NetworkX):

- DataFrames: `apps`, `qmgrs`, `queues`, `channels`, `relations`.
- Graph: `G` with nodes = QMgrs, edges = channels.

Processing steps:

1. Load CSVs → build DataFrames.
2. Build graph `G`.
3. Run **metric functions** to compute TCS(as-is).
4. Apply **rules in sequence**, each returning:
   - updated DataFrames / graph
   - a list of `{rule_id, object_id, action, reason}` for explainability.
5. Recompute metrics → TCS(target).
6. Export:
   - target CSVs,
   - changes report,
   - metrics (before/after) table.

If you like, next I can draft concrete rule pseudocode (or Python function signatures) that you can drop into your project.